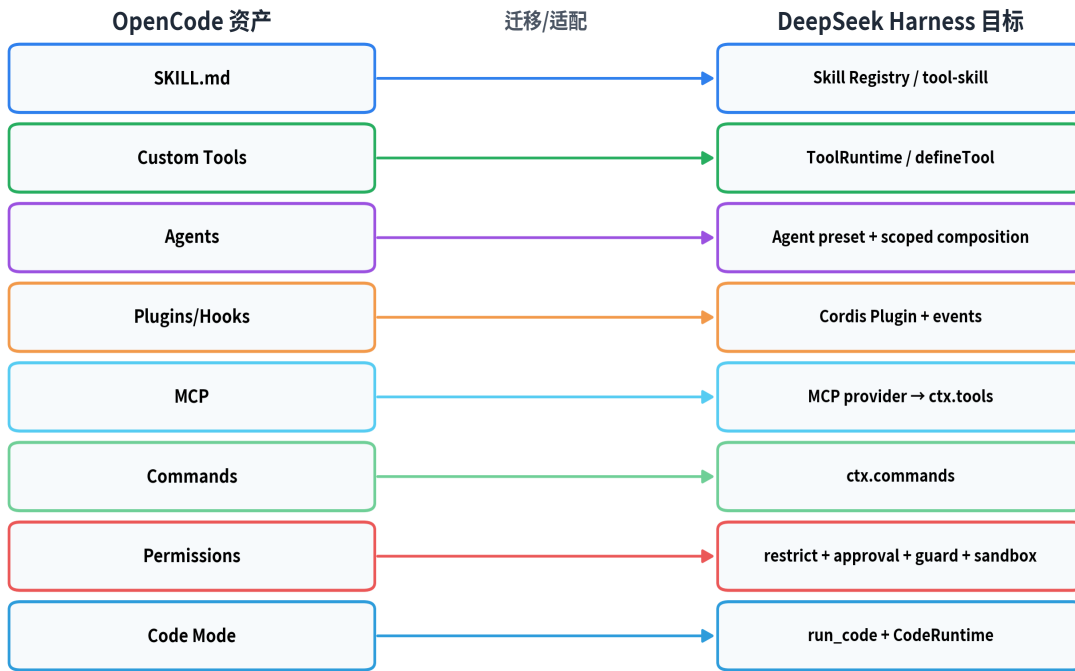


OpenCode → DeepSeek Harness

Skill / Tool / Agent / Plugin / MCP 综合迁移计划与工程实施指南

OpenCode → DeepSeek Harness

Skill / Tool / Agent / Plugin / MCP 综合迁移计划与工程实施指南



核心原则：迁移“能力契约”，而不是复制目录结构

图 1 OpenCode 资产到 DeepSeek Harness 能力契约的迁移总览

核心判断：不要做“目录级搬家”。应建立一层可审计的 Migration Compiler，把 OpenCode 资产编译为 Harness 的 Skill、ToolRuntime、Agent/Subagent、Cordis Plugin、Policy 和 Workflow 能力。

报告性质：源码级迁移设计 / 企业内部工程落地 / 兼容性与风险治理

0. 执行摘要

如果目标是在公司内部逐步把 OpenCode 上已经积累的 Skills、Tools、Agents、Plugins、MCP 与权限配置迁移到 DeepSeek Harness，最危险的做法是直接做“一次性重写”。两套系统虽然都属于 coding-agent / agent-harness，但扩展点的边界不同：OpenCode 更偏“产品内置

Agent + 配置/插件扩展”，DeepSeek Harness 则把 Session、Tool、Agent Loop、LLM Adapter、Sandbox 等都作为 Cordis 可替换能力。迁移的核心不是语法翻译，而是把已有资产重新落到更清晰的能力契约上。

推荐策略：采用 7 个迁移波次：Inventory → Skills → Tools → Agents/Permissions → Plugins/Hooks → PTC/Workflow → Cutover。每一波都保留 OpenCode 作为可回滚基线，并使用同任务 Shadow Run 做功能、安全和体验等价性验证。

资产	推荐目标	直接可搬?	主要差异	优先级
SKILL.md	ctx.skills + dsh-tool-skill	高	目录/Frontmatter 小差异；.agents/skills 可直接复用	P0
Custom Tool	ctx.tools.register / defineTool	中	Zod→Harness Schema；Harness 强制 canonical output schema	P0
Agent	Agent preset / scoped composition	低-中	OpenCode Agent 配置不是 Harness preset 的同构格式	P1
Subagent	ctx.subagents + toolFilter/persona/provider	中	Harness 将 subagent 做成 capability seam，可多 provider 共存	P1
Plugin/Hook	Cordis Plugin + typed events	低	事件模型和生命周期模型不同，需重构而非机械翻译	P1
MCP	MCP plugin→ctx.tools	高	协议相同，重点是权限、命名、输出 Schema 与审计	P0
Permission	restrict + approval + guard + sandbox	低	OpenCode 单一 permission ruleset 要拆成多层治理	P0
Code Mode	run_code + CodeRuntime	低	OpenCode 当前 execute 更偏 MCP orchestration；Harness 面向整个 ToolRuntime	P2
Session History	Session/Event import（可选）	低	数据模型差异大，不建议作为第一阶段目标	P3

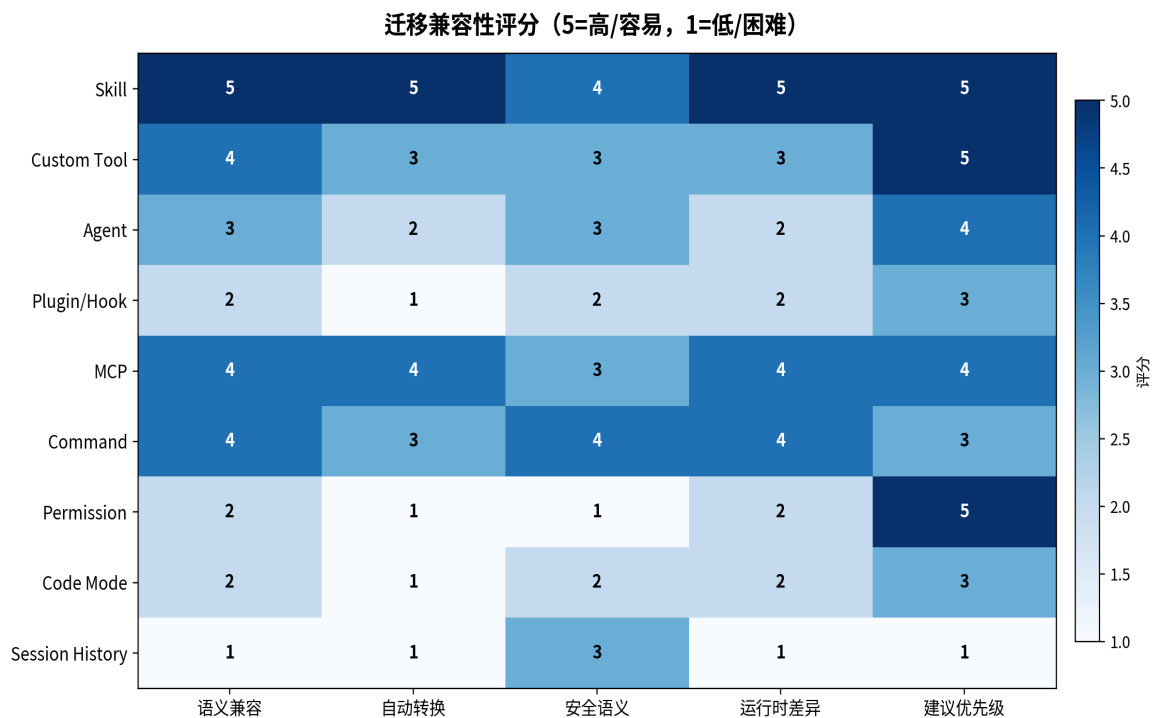
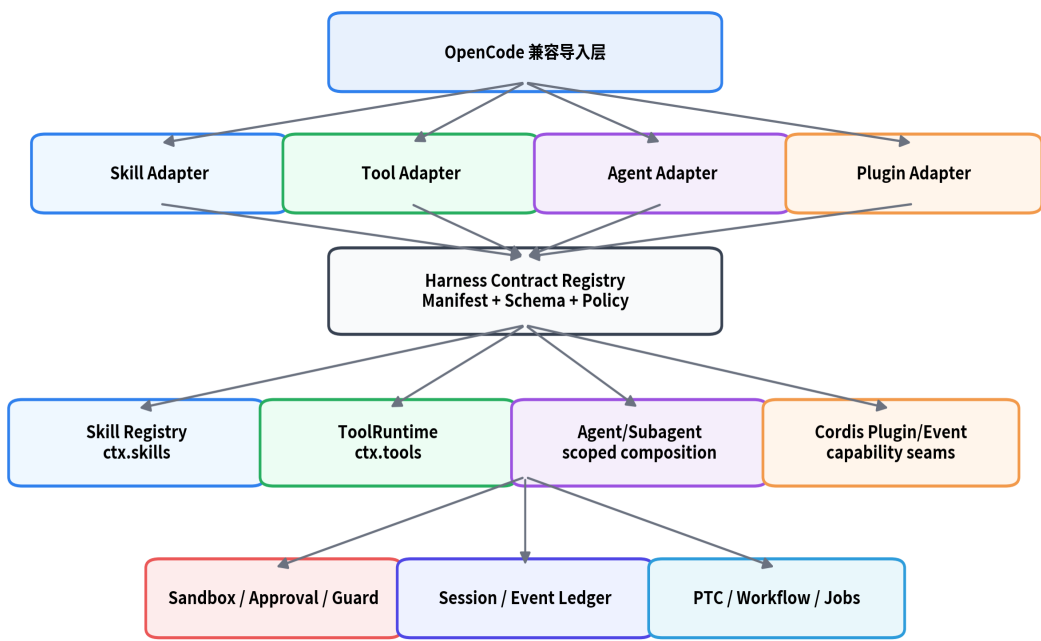


图 2 迁移兼容性评分：Skill 与 MCP 最适合先迁，Permission/Plugin/历史 Session 最需要重新设计

0.1 最终建议的企业目标架构

建议在 DeepSeek Harness 之上增加一个公司内部 Migration/Compatibility Layer，而不是长期维护一套 OpenCode Runtime 兼容层。兼容层只负责“解析与编译”，执行仍统一进入 Harness。



目标：把 OpenCode 资产转成稳定的 Harness 能力契约，而不是形成第二套兼容运行时

图 3 建议的目标架构：兼容层只负责导入，运行时统一收敛到 Harness 能力契约

1. 调研范围与源码基线

本报告按 2026-08-18 可见的上游源码/文档进行设计。OpenCode 以 anomalyco/opencode 的 dev 分支为基线；DeepSeek Harness 以 deepseek-ai/deepseek-harness 的 master 为基线。两边均处于快速演进中，迁移器必须显式记录 source version / commit / schema version，不能把“当前行为”当作永久 API。

系统	核心事实	迁移意义
OpenCode	Skills 通过 SKILL.md 按需加载；自定义 Tool 由 @opencode-ai/plugin 的 tool() 定义；Agent 支持 primary/subagent；Permission 使用 allow/ask/deny + pattern；Plugin 可挂钩工具/会话/压缩等事件。	现有资产通常是“配置 + JS/TS 执行体 + 权限规则”。
DeepSeek Harness	Everything is a Plugin；Cordis 驱动；Session 为 append-only event log；ToolRuntime 有 schema/guard/pipeline；Skill 是 provider registry；Subagent/Workflow 是独立 capability seam；Sandbox fail-closed。	目标不是复制产品层配置，而是映射到 capability seam + scoped composition。

1.1 OpenCode 资产面

- ``.opencode/skills/<name>/SKILL.md``、``.~/config/opencode/skills``、``.agents/skills`` 等技能目录；
- ``.opencode/tools/*.ts`` / ``.~/config/opencode/tools/*.ts`` 自定义工具；
- ``.opencode.json`` 与 ``.opencode/agents/*.md`` 的 Agent 配置；
- ``.opencode/plugins/*`` 与 npm plugins 的 Hook/Tool 扩展；
- MCP server 配置与其暴露的工具；
- Permission ruleset（工具名/路径/命令 pattern → allow/ask/deny）；
- Code Mode ``.execute``：当前源码侧重把权限可见 MCP tools 放入受限脚本解释器。

1.2 DeepSeek Harness 目标面

- ``.ctx.skills``：Skill provider registry；filesystem provider 原生扫描 ``.dsh/skills`` 与 ``.agents/skills``；
- ``.ctx.tools``：统一 ToolRuntime；``.defineTool()`` 负责 typed parameters + mandatory canonical output schema；
- Agent preset / scoped composition：通过 agent scope 覆盖工具、prompt、provider 与 plugin 贡献；
- ``.ctx.subagents``：多 provider 共存，可声明 persona、toolFilter、maxDepth、outputSchema 等能力；

- `ctx.workflowEngine`：模型生成脚本编排 subagents；
- Cordis Plugin/Event：服务、typed events 与 reversible effects 组成可替换能力；
- Sandbox / Approval / Guard：权限、审批、不可逆拒绝和文件执行隔离分层治理；
- Session/Event Ledger：模型可见历史从 durable events 投影，而不是仅存 messages。

2. 迁移设计原则

P1：能力契约优先：迁移的是“这个资产能做什么、输入输出是什么、权限是什么、生命周期是什么”，不是原目录和类名。

P2：Fail Loud：无法无损映射的字段直接进入 migration_report 的 manual_review，不允许静默丢弃。

P3：安全不降级：OpenCode 中 deny/ask 的能力，迁移后不能变成默认 allow；Harness 的 sandbox/approval/guard 应显式配置。

P4：先兼容再重构：Tool 首轮可以先输出 string canonical value，保证行为等价；第二轮再升级成结构化 output schema。

P5：单一执行内核：兼容层不自行执行 Tool/Skill；所有实际执行必须进入 Harness ToolRuntime/Skill/Subagent 等正式 seam。

P6：可回滚：每个 asset 都保留 source hash、generated files 和 migration manifest，允许一键回到 OpenCode。

P7：双跑验证：相同任务在 OpenCode 与 Harness 运行，比较最终结果、工具轨迹、权限询问、时间、token、错误与修改 diff。

3. Wave 0：资产盘点与依赖图

迁移开始前先生成机器可读的 inventory.json。不要先改代码。Inventory 是后续转换、风险判断、灰度和回滚的唯一索引。

```
{
  "source": {"runtime": "opencode", "version": "...", "commit": "..."},
  "assets": [
    {
      "id": "tool:database",
      "kind": "tool",
      "path": ".opencode/tools/database.ts",
      "imports":["@opencode-ai/plugin"],
      "externalExecutables":["python3"],
```

```
"permissions":["database"],
"usedByAgents":["build","data-reviewer"],
"risk":"medium",
"migrationClass":"tool-v1"
}
]
}
```

3.1 Inventory Scanner 应扫描什么

对象	扫描内容	为什么重要
Skills	路径、name/description/ frontmatter、引用的 scripts/ assets/references	判断是否能直接放到 .agents/ skills；识别运行时依赖
Tools	tool name、args schema、 execute body、外部进程、文件/网络访问、返回类型	决定 defineTool schema 和 sandbox/approval
Agents	mode、model、prompt、steps、 permission、skills/tools 依赖	生成 Harness preset/persona/ toolFilter
Plugins	注册事件、注入工具、环境变量、 compaction/session hook	确定映射到哪类 Cordis event/ service
MCP	server、transport、tool names、 schema、认证依赖	检查命名冲突与权限编译
Permissions	global rules + per-agent override + bash/path patterns	构建 Harness policy compiler
Commands	命令名、模板、是否需要模型 turn	映射 ctx.commands 或 Skill/ Workflow
Code Mode	脚本是否调用 MCP-only namespace；是否依赖 OpenCode 特定 interpreter	决定重写为 run_code 或 Workflow

3.2 迁移分级

级别	定义	动作
A - 直接兼容	仅路径/Frontmatter 差异	自动转换并自动测试
B - 结构转换	Schema/配置结构不同，但执行语义明确	自动生成 + 人工 review
C - 运行时转换	依赖 OpenCode context/hook/ plugin lifecycle	生成 skeleton + 手工重构
D - 语义重构	权限、编排、session/history、 Code Mode 语义不同	重新设计，不做机械翻译

4. Wave 1: Skills 迁移（优先级最高、风险最低）

OpenCode 与 DeepSeek Harness 在 Skills 上兼容性非常高。OpenCode 原生支持 ``.agents/skills/<name>/SKILL.md``；Harness 的 filesystem skill provider 也把 ``.<projectRoot>/.agents/skills`` 作为默认扫描根之一。因此最优策略不是转换内容，而是先把 Skill 资产统一到 ``.agents/skills`` 作为跨 Harness 的共享格式。

最快落地：如果现有 Skill 已经位于 ``.agents/skills`` 且 frontmatter 至少有合法 ``name`` 与 ``description``，可以先不搬文件，仅启动 Harness 验证是否可发现。

OpenCode Frontmatter	Harness 对应	迁移规则
name	name	直接保留；kebab-case 校验
description	description	直接保留
license	无核心执行语义	保留在源文件/metadata；不作为 runtime policy
compatibility	无直接执行语义	迁移报告记录；必要时转 metadata
metadata	metadata	保留，但注意目标 provider 的类型约束
-	whenToUse	可从 Skill 正文 “When to use” 自动提取，建议人工确认
-	disable-model-invocation	公司内部可用于 human-only Skill
-	user-invocable	控制人类入口；默认 true

4.1 推荐的 Skill 目录

```
repo/
.agents/
skills/
git-release/
SKILL.md
scripts/
references/
assets/
.dsh/
# 只放 Harness 特有配置，不复制 Skills
```

4.2 Skill 迁移验收

- OpenCode 与 Harness 的 skill catalog 都能看到相同 name/description；

- 模型加载同一个 Skill 时，正文语义一致；
- 被 deny/disabled 的 Skill 不应在目标模型可见 catalog 中出现；
- Skill 引用脚本时，不能假设用户 Host 环境一致：优先改为 Tool/PTC/统一 runtime，而不是把环境安装逻辑塞进 Skill。

5. Wave 2: Custom Tools 迁移

Tool 是第一批真正需要“编译”的资产。OpenCode 自定义 Tool 使用 `tool()` + Zod 定义参数，执行函数通常直接返回字符串；Harness `defineTool()` 则要求参数 schema 与 mandatory `output.schema + render`，并把 canonical output 与模型可见 presentation 分离。迁移器必须显式处理这个差异。

维度	OpenCode	DeepSeek Harness	迁移动作
参数 Schema	tool.schema / Zod	ParameterSchemaSpec / raw JsonSchemaNode	做 Zod subset → Harness schema 转换
输出	常见为 string / ExecuteResult	mandatory canonical output schema	首轮 string；二轮结构化
Context	agent/sessionID/messageID/directory/worktree	ToolExecution + agent/session services + exec.signal	建立 context adapter
权限	permission ruleset	pre-execute + approval + guard + sandbox	不能直接等价复制
并发	工具实现自行决定	isConcurrencySafe(args) + ordered scheduler	显式声明 read-only 工具
UI	title/output/metadata/attachments	presentCall/presentResult + render	可做二期体验适配

5.1 推荐转换：先“行为兼容”，再“类型升级”

```
// OpenCode
export default tool({
  description: "Query DB",
  args: { query: tool.schema.string() },
  async execute(args) {
    return await runQuery(args.query) // string
  }
})

// Harness v1 compatibility wrapper
ctx.tools.register(defineTool({
  name: "database",
  description: "Query DB",
```



```
parameters: { query: { type: "string", required: true } },
output: {
schema: { type: "string" },
render: (_args, value) => [{ type: "text", text: value }]
},
async execute(args, exec) {
return await runQuery(args.query, exec.signal)
}
}))
```

二期升级：把数据库/搜索/API Tool 从“返回一段字符串”升级为 canonical JSON，例如 `{items,total,cursor}`，再由 render 生成模型文本。这样 PTC、Workflow、评测和 UI 才能共享真正的结构化数据。

5.2 Zod → Harness Schema 支持策略

Zod 形态	自动转换建议	处理
string/number/boolean	100% 自动	直接映射
enum/literal	自动	映射 enum/const
array/object	自动或半自动	显式决定 additionalProperties
optional	自动	不进入 required
union	有限支持	转 exactly-one oneOf 前检查歧义
refine/transform/superRefine	不可机械等价	manual_review + 执行前 validator
date/bigint/class instance	不属于 lossless JSON	重新设计为 string/number/object wire type
default	不要假设 Harness 会注入默认值	在 execute/adaptor 显式处理

6. Wave 3：Agents、Subagents 与权限迁移

OpenCode 的 Agent 是产品级配置单元：primary/subagent + model + prompt + steps + permission。Harness 更强调 “Agent + scoped composition”：同一 Tool Registry/Skill Registry 在 agent scope 中可以被 restrict、shadow 或添加局部能力，Subagent 又是独立 capability seam。因此推荐把 OpenCode Agent 编译成一个 “Agent Manifest”，再由 Harness preset builder 生成实际 composition。

```
agent-manifest.yaml
name: review
mode: subagent
```

persona: ./prompts/review.md

model: iagent/standard

limits:

maxSteps: 8

capabilities:

tools:

deny: [write, edit, bash]

skills:

allow: [code-review, security-review]

sandbox:

mode: read-only

approval:

default: deny

6.1 Agent 字段映射

OpenCode Agent 字段	Harness 目标	说明
description	preset/descriptor description	用于发现/选择
mode=primary	profile/preset 的主要交互 Agent	不建议硬编码成某个 core loop fork
mode=subagent	ctx.subagents provider + persona/ toolFilter	可选择 spawn/fork/ACP/外部 provider
model	LLM adapter/preset override	通过 provider/model policy 映射
prompt	persona / system-prompt section	保留模板变量时需做兼容审计
steps	Agent Loop / deployment limit	如果目标没有同构字段，用 stop policy 实现
permission	policy compiler	拆成 visibility/approval/guard/ sandbox 四层
tools(deprecated)	tools.restrict	仅作为 visibility，不作为最终安全 边界

6.2 Permission 不能一对一复制

OpenCode 的 permission 规则把“是否可见、是否询问、是否允许某些 bash/path pattern”集中在一个 ruleset；Harness 更适合拆层。迁移器必须把每条 OpenCode permission 编译到最合适的目标层，而不是只做 `tools.restrict()`。

OpenCode 语义	Harness 目标层	迁移原则
deny 某工具	tools.restrict + guard	从模型视野移除，同时防止绕过式 执行
ask 某工具	tools/pre-execute → approval	进入统一审批 UI/服务

OpenCode 语义	Harness 目标层	迁移原则
allow 某工具	visibility 保留 + policy allow	不能自动推出可写文件系统
bash command pattern	shell/tool policy guard	用解析后的 command/argv 做规则，不要仅字符串 contains
外部目录读写	sandbox workspace boundary + fs policy	Harness 的 workspace-write/read-only 更适合做文件效果边界
高风险写操作	approval + exclusive concurrency	避免并发修改并保留审计顺序

安全底线：如果 OpenCode 原规则为 deny/ask，而目标 Harness 无法表达等价约束，该资产必须标为 BLOCKED，不能自动降级成 allow。

7. Wave 4: Plugins / Hooks → Cordis Plugins

这是迁移难度最高的常规资产。OpenCode Plugin 以 hook/event 回调扩展产品；Harness 的 Cordis 把服务、事件和 reversible effects 作为组合基础。迁移应先按“Hook 意图”分类，再选择 Harness extension point。

OpenCode Plugin 意图	Harness 建议扩展点	备注
注册自定义 tool	ctx.tools.register	最直接；Tool schema 会自动进入 prompt assembly
tool.execute.before	tools/pre-execute 或 ctx.tools.guard	可改写/审批 vs monotonic denial 需要区分
tool.execute.after	tools/post-execute / tools/result	前者可变换，后者只观察
包裹执行/重试/timeout/metrics	tools/execute around wrapper	比 before/after 更接近中间件
Session 观察	session/event	durable event 与 live agent event 不要混用
请求/Prompt 改写	agent/request / system-prompt assembly	必须保证模型可见内容可从 Session 重建
Compaction hook	compaction subsystem events	不要把摘要直接覆盖原始事件日志
日志/遥测	telemetry capability seam	不要靠 console.log 作为生产观测
生命周期副作用	Cordis reversible effect	卸载时应精确释放注册/资源

7.1 自动迁移器的边界

- 可以自动识别 `tool()` 注册、静态 args schema、简单 before/after hook；
- 不能自动证明 hook 顺序语义、跨 hook 共享状态、全局 mutable state、网络/文件副作用是否安全；
- 对于复杂 plugin，应生成 Cordis Plugin skeleton + TODO + 原源码引用，而不是生成“看似能跑”的错误实现。

8. MCP 与 Code Mode 迁移

8.1 MCP：协议兼容高，但治理要重做

MCP 是较适合自动迁移的部分。Harness Tools 文档明确把 MCP server 作为一种 Tool Plugin：发现 schema 后调用 `ctx.tools.register()`。但迁移时仍要重新处理命名冲突、approval、sandbox、认证注入、output schema 与结果大小。

检查项	迁移要求
Tool name	建立 stable namespace，避免 OpenCode 的 <code>`server_tool`</code> 与 Harness 内置工具碰撞
Input schema	验证是否落在 Harness 支持的 JSON Schema subset；必要时 adapter normalize
Output schema	MCP 若缺失 output schema，生成保守 <code>`json`/string</code> 兼容层并记录 tech debt
Auth	不要把密钥写入 Skill/Prompt；交给 credential/provider 层
Permission	MCP server 级规则编译为 scoped visibility + approval
Large result	做 spill/locator 或摘要策略，避免模型 context 被原始 payload 撑爆

8.2 OpenCode Code Mode 与 Harness PTC 不应直接互转

当前 OpenCode ``code-mode.ts`` 的 ``execute`` 会按当前 permission 可见性收集 MCP tools，构造受限 interpreter tool tree，再执行脚本；Harness Code Mode 则是 ToolRuntime 的 presentation mode：``code`` 模式把整个可见工具集投影为 ``run_code` + generated SDK``，每个 ``tools.xxx()`` 仍重新进入统一 ToolRuntime policy pipeline。

维度	OpenCode 当前 Code Mode	Harness Code Mode
模型入口	<code>execute({code})</code>	<code>run_code({code,description})</code>
主要工具面	当前源码聚焦 MCP tools	所有当前 agent 可见 ToolRuntime tools
权限	先按 permission 过滤 + 子调用 ask	每个 binding 子调用重新进入统一 ToolRuntime/guard/approval
输出	解释器 result/logs/attachments	outer logs/return；inner dispatch 只进事件日志
迁移策略	把脚本意图提取成测试 fixture	重新生成 Harness SDK 下的 program，不做字符串替换

9. 推荐的迁移后仓库结构

repo/

.agents/
skills/ # 跨 Agent 兼容 Skills
.dsh/
migration/
manifest.json # 资产来源、hash、转换状态
reports/ # manual_review / incompatibility
fixtures/ # 双跑测试任务
plugins/
company-tool-pack/ # Cordis Tool plugins
company-policy/ # approval / guards / restrictions
company-agent-presets/ # agent scoped composition
config/
agent-manifest.yaml
policy-map.yaml
packages/
tool-contracts/ # 可选: 公司统一 Tool IDL/Schema
migration-compiler/ # OpenCode parser → Harness generator
tests/
migration/
skills/
tools/
agents/
security/
shadow/

9.1 Migration Manifest

```
{  
  "assetId": "tool:database",  
  "source": {  
    "runtime": "opencode",  
    "path": "opencode/tools/database.ts",  
    "sha256": "..."  
  },  
  "target": {  
    "runtime": "dsh",  
    "kind": "tool-plugin",
```

```
"package": "company-tool-pack",
"name": "database"
},
"conversion": {
  "status": "generated-with-review",
  "warnings": ["output inferred as string", "zod refine not portable"]
},
"tests": {
  "contract": "pass",
  "shadow": "pass",
  "security": "pass"
}
}
```

10. Migration Compiler 设计

推荐做成独立 CLI，而不是一次性脚本。它不负责运行 OpenCode，也不负责运行 Harness；只做 parse → normalize → validate → generate → report。

```
opencode-migrate scan ./repo
opencode-migrate plan ./repo --target dsh
opencode-migrate generate ./repo --target dsh
opencode-migrate validate ./repo
opencode-migrate shadow ./fixtures --source opencode --target dsh
opencode-migrate report --format html,pdf,json
```

10.1 编译流水线

阶段	输入	输出
Parse	OpenCode JSON/Markdown/TS static AST	SourceAsset IR
Normalize	SourceAsset IR	Canonical Migration IR
Policy Compile	OpenCode permissions	Visibility/Approval/Guard/Sandbox IR
Schema Compile	Zod/JSON Schema	Harness Parameter/Value Schema
Generate	Canonical IR	Skill files / Tool plugins / preset skeletons
Static Validate	Generated source	schema/load/config diagnostics
Dynamic Validate	Harness test profile	tool/skill/agent smoke results

阶段	输入	输出
Shadow Eval	common fixtures	equivalence metrics + diff

11. 验证体系：迁移完成的定义



“能运行”不是迁移完成；真正的完成标准是功能、安全、可观测与使用体验同时达标

图 4 迁移验收漏斗：只有“生产灰度”通过才算迁移完成

维度	核心指标	建议 Gate
解析覆盖率	被 scanner 识别的资产 / 总资产	>= 99% 或全部未识别项有 owner
自动转换率	无需人工改写即可生成目标资产	Skill >90%, Tool >70%, Plugin 不设硬目标
契约正确率	Schema compile / load 成功	100%
功能等价	同任务最终结果与关键 side effects	关键任务 100%，普通任务 >=95%
Tool 轨迹	工具集合、参数有效率、重试次数	无系统性恶化
安全等价	deny/ask/sandbox regression	0 个权限放大
性能	P50/P95 latency、token、tool call 数	按业务设阈值
体验	人工 approval 数、错误可理解度、恢复能力	关键用户 UAT 通过

11.1 Shadow Run 数据结构

```
{
  "fixture":"repo-refactor-017",
  "source":"opencode",
```

```
"target": "dsh",
"assertions": {
  "filesChanged": ["src/a.ts", "src/b.ts"],
  "testsPass": true,
  "forbiddenPathsUntouched": true
},
"metrics": {
  "wallMs": 0,
  "modelCalls": 0,
  "toolCalls": 0,
  "approvalPrompts": 0,
  "inputTokens": 0,
  "outputTokens": 0
}
}
```

12. 分阶段迁移路线图

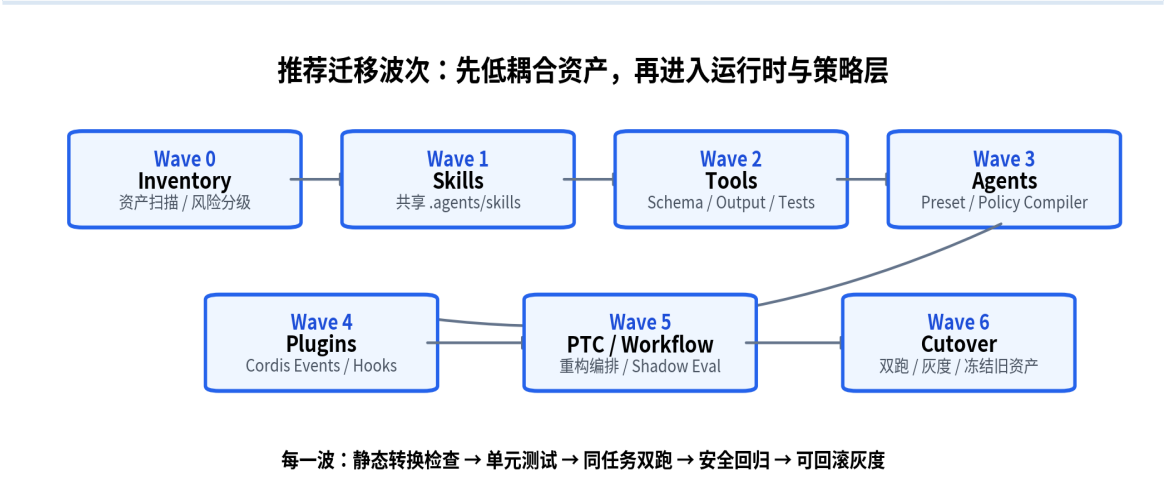


图 5 推荐迁移波次

Wave	主要工作	退出条件
0 Inventory	扫描所有 OpenCode 资产，建立依赖图/风险级别	100% 资产有 owner 与 migration class
1 Skills	统一 ``.agents/skills``；修 frontmatter；Skill catalog 双跑	核心 Skill 在 Harness 可发现/加载
2 Tools	Zod→Schema、string wrapper、Tool contract test	P0 Tools 功能与安全等价
3 Agents/Permission	生成 Agent Manifest；编译 visibility/approval/sandbox	Build/Plan/关键 Subagent 可替代

Wave	主要工作	退出条件
4 Plugins/Hooks	复杂 Hook 重构为 Cordis Plugin/Event	核心生命周期扩展覆盖
5 PTC/Workflow	选择高收益批处理/多 Agent 任务重写	Shadow Eval 明显优于或不劣于 OpenCode
6 Cutover	双运行、分团队灰度、冻结新 OpenCode 资产	回滚机制验证后正式切换

13. 灰度与回滚设计

迁移期建议“资产级路由”而不是“用户级二选一”。同一个用户可以让已迁移的 Skill/Tool 走 Harness，尚未迁移的复杂 Plugin 继续留在 OpenCode。最终才切产品入口。

模式	适用阶段	实现
Read-only shadow	Wave 1-2	Harness 不写文件，仅对比理解/检索结果
Tool shadow	Wave 2	对纯读取 Tool 双执行，canonical output 做 diff
Dry-run write	Wave 2-3	目标 Tool 生成 patch/diff，不真正提交
Team canary	Wave 3-5	少量团队默认 Harness，可一键回 OpenCode
Asset freeze	Wave 6	禁止新增仅 OpenCode 的 Skill/Tool；CI 阻止技术债继续扩大
Final cutover	Wave 6	OpenCode 只保留 read-only fallback 窗口

14. 关键 Gotchas 与风险清单

Skill 目录看似兼容，但 invocation policy 不完全相同：不要只看文件是否能加载，要验证模型可见/用户可见与 deny/ask 语义。
OpenCode Tool 通常没有强制 output schema：如果直接全部包装成 string，会失去 Harness PTC/结构化评测价值；只作为过渡。
OpenCode Agent permission 不是 Harness sandbox：“tool 不可见”与“进程无法写工作区外”是不同安全层。
Plugin hook 顺序与 Cordis waterfall/emit 不同：必须逐个确认能否 rewrite、block、observe，以及 listener failure 是否应传播。
Code Mode 同名但语义不同：不要直接迁脚本；迁移任务意图和 fixture。

- Subagent 并非同一个生命周期模型：Harness 有 durable child session / continuable activation / provider capabilities；需要重新决定 child ownership。
- 动态 Tool/Skill 会影响 prompt/KV cache：迁移后要控制注册顺序、Skill catalog 变更频率、Tool schema 稳定性。
- Host 环境差异：把 OpenCode Tool 中直接调用 python/go/node 的脚本迁到受控 runtime/container，避免用户环境崩溃。
- Session 历史不应第一阶段强迁：两边数据模型差异大；优先迁能力，历史只做只读归档或选择性导入。
- Harness Developer Preview：所有生成物必须带 source version，并设置 nightly compatibility test。

15. 推荐的 P0 实施清单

1. 建立 `migration-compiler` 仓库和 Canonical Migration IR；
2. 扫描所有 `.opencode/skills`、`.agents/skills`、`.opencode/tools`、`.opencode/agents`、plugins 与 `opencode.json`；
3. 先把 Skills 统一到 `.agents/skills`，不改正文；
4. 挑 10 个最高频 read-only Tools 迁为 Harness `defineTool()`；
5. 为每个 Tool 补 `output.schema`、contract tests 与 `isConcurrencySafe`；
6. 实现 OpenCode permission → Harness Policy IR 编译器，默认 fail closed；
7. 先重建 `build` / `plan` / `general(explore)` 三类 Agent，不追求一次覆盖所有自定义 Agent；
8. 建立 30-50 个真实任务 fixtures，同任务双跑；
9. 先在一个团队 Canary；
10. 达到 Gate 后再迁复杂 plugins、Code Mode 和 Workflow。

16. 示例：一个真实资产包如何迁移

假设 OpenCode 项目里有 `review` Agent、`security-review` Skill、`gitlab` Tool、一个 tool.execute.before 审批 Plugin，以及 GitLab MCP。推荐拆成以下目标：

源资产	目标 Harness 资产	原因
.opencode/skills/security-review/SKILL.md	.agents/skills/security-review/SKILL.md	跨平台共享，最低改造
.opencode/tools/gitlab.ts	company-tool-pack: gitlab.* ToolDefinition	统一 input/output schema 和 policy
review.md Agent	review Agent Manifest + persona + toolFilter + read-only sandbox	把角色、能力和安全分开

源资产	目标 Harness 资产	原因
before hook 审批	tools/pre-execute + approval plugin	使用统一审批通道
GitLab MCP	MCP provider → ctx.tools namespace	保留协议，统一工具治理
多 API 批量分析脚本	run_code PTC	在运行时做 filter/map/parallel，减少 LLM round-trip

17. 企业治理与长期维护

治理对象	建议
Source of Truth	Skill 用 <code>`.agents/skills`</code> ；Tool 用公司 Tool Contract/IDL；Agent 用 Agent Manifest，不同时维护两套配置
Versioning	每个生成资产记录 source runtime + commit + converter version
CI	上游 OpenCode/DSH 更新后自动跑 parser compatibility + smoke + security fixtures
Ownership	每个 Tool/Skill/Plugin 绑定 owner/team；manual_review 不允许无人负责
Security	所有 privilege expansion 需要 code owner 审批；migration diff 单独展示 policy 变化
Deprecation	Cutover 后 <code>`.opencode`</code> 进入冻结/只读；不允许新资产只落旧平台
Telemetry	同时记录 tool error rate、approval、sandbox deny、latency、token、context compaction 和 user fallback

18. 结论

从迁移成本看，最值得先做的是 Skill、MCP 与高频 Tools；最不应该先做的是复杂 Plugin 和历史 Session。真正决定迁移是否成功的不是“有多少文件转过去”，而是能否把现有资产收敛成 Harness 的统一能力契约：ToolRuntime 负责工具执行，SkillRegistry 负责指令资产，Agent/Subagent 负责角色和委派，Cordis Plugin 负责生命周期扩展，Sandbox/Approval/Guard 负责安全，Session/Event 负责可重放轨迹。

最终建议：把迁移项目定义为“OpenCode Asset Compiler → DeepSeek Harness Capability Platform”，而不是“OpenCode 配置转换器”。前者能形成公司长期可维护的 Agent 基础设施；后者很容易随着两边版本演进迅速失效。

19. 主要源码与文档参考

- **DeepSeek Harness README:** <https://github.com/deepseek-ai/deepseek-harness/blob/master/README.md>
- **DeepSeek Harness Architecture:** <https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/architecture.md>
- **DeepSeek Harness Tools:** <https://github.com/deepseek-ai/deepseek-harness/blob/master/packages/core/tools/README.md>
- **DeepSeek Harness Skill Registry:** <https://github.com/deepseek-ai/deepseek-harness/blob/master/packages/skill/skill/README.md>
- **DeepSeek Harness Skill Filesystem:** <https://github.com/deepseek-ai/deepseek-harness/blob/master/packages/skill/skill-filesystem/README.md>
- **DeepSeek Harness Subagent:** <https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/subsystems/subagent.md>
- **DeepSeek Harness Workflow:** <https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/subsystems/workflow.md>
- **DeepSeek Harness Sandbox:** <https://github.com/deepseek-ai/deepseek-harness/blob/master/docs/subsystems/sandbox.md>
- **OpenCode README:** <https://github.com/anomalyco/opencode/blob/dev/README.md>
- **OpenCode Skills:** <https://github.com/anomalyco/opencode/blob/dev/packages/web/src/content/docs/skills.mdx>
- **OpenCode Agents:** <https://github.com/anomalyco/opencode/blob/dev/packages/web/src/content/docs/agents.mdx>
- **OpenCode Custom Tools:** <https://github.com/anomalyco/opencode/blob/dev/packages/web/src/content/docs/custom-tools.mdx>
- **OpenCode Plugins:** <https://github.com/anomalyco/opencode/blob/dev/packages/web/src/content/docs/plugins.mdx>
- **OpenCode Permission Source:** <https://github.com/anomalyco/opencode/blob/dev/packages/opencode/src/permission/index.ts>
- **OpenCode Code Mode Source:** <https://github.com/anomalyco/opencode/blob/dev/packages/opencode/src/tool/code-mode.ts>

注：两项目均快速迭代；实施迁移时应把上述 URL 替换/固定到实际使用的 commit SHA，并将该 SHA 写入 Migration Manifest。